

Session 2

Sensing our world

Important terms:

VCC = Power = either 5V or 3.3V = Positive(+) end of power source

GND = Ground = 0V = Negative(-) end of power source

I²C = requires 4-pins: SDA, SCL, VCC, GND

Working with ESP32

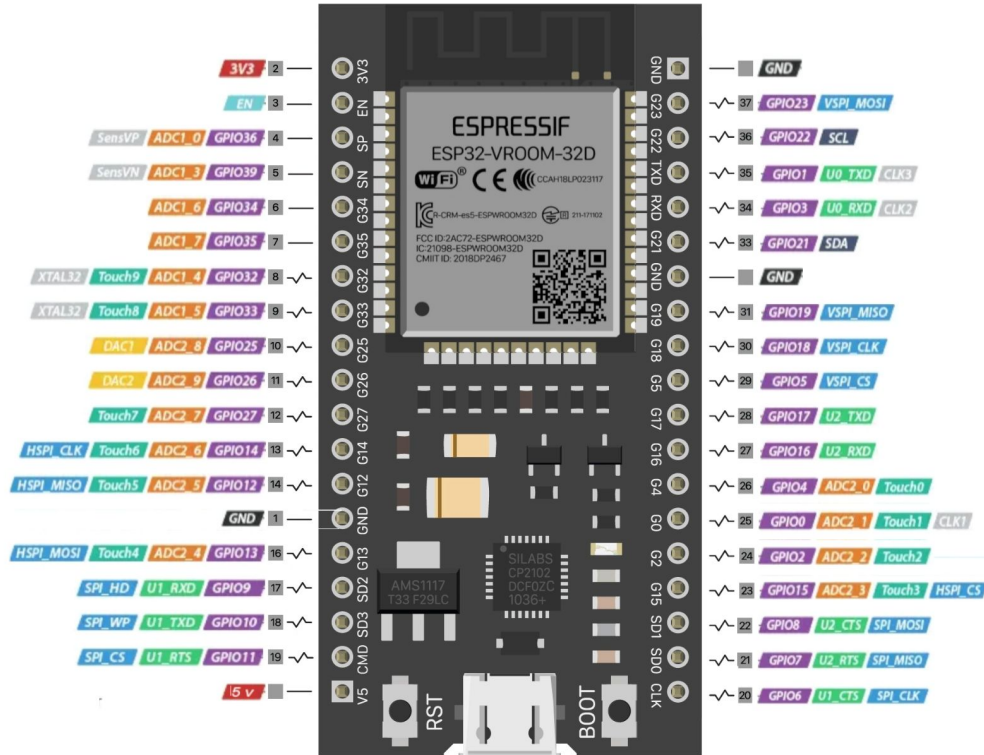
I²C (Inter-Integrated Circuit) is a two-wire communication protocol for connecting sensors, using just two lines: SCL (clock) and SDA (data).

Addressing (unique IDs for each device) allow multiple devices to share the same lines, controlled by a microcontroller

I²C

SDA: 21

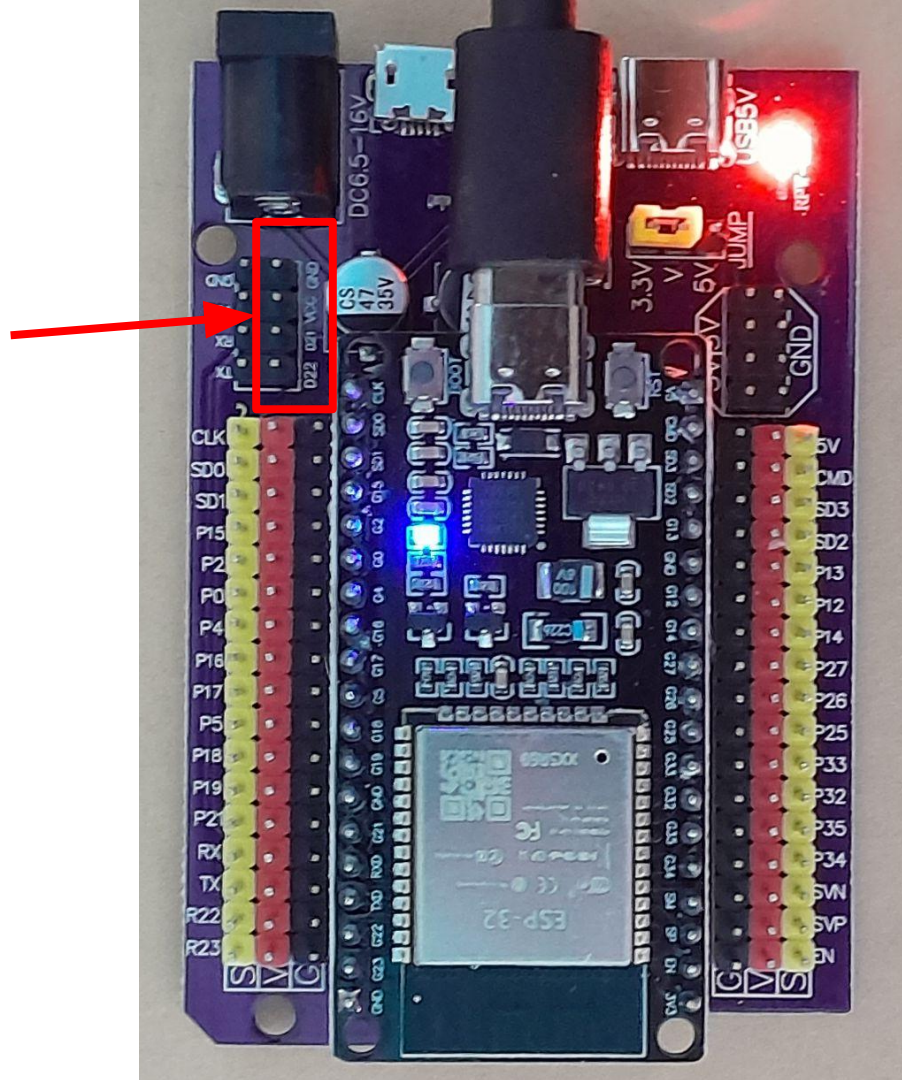
SCL: 22



Working with ESP32

Esp32 breakout board

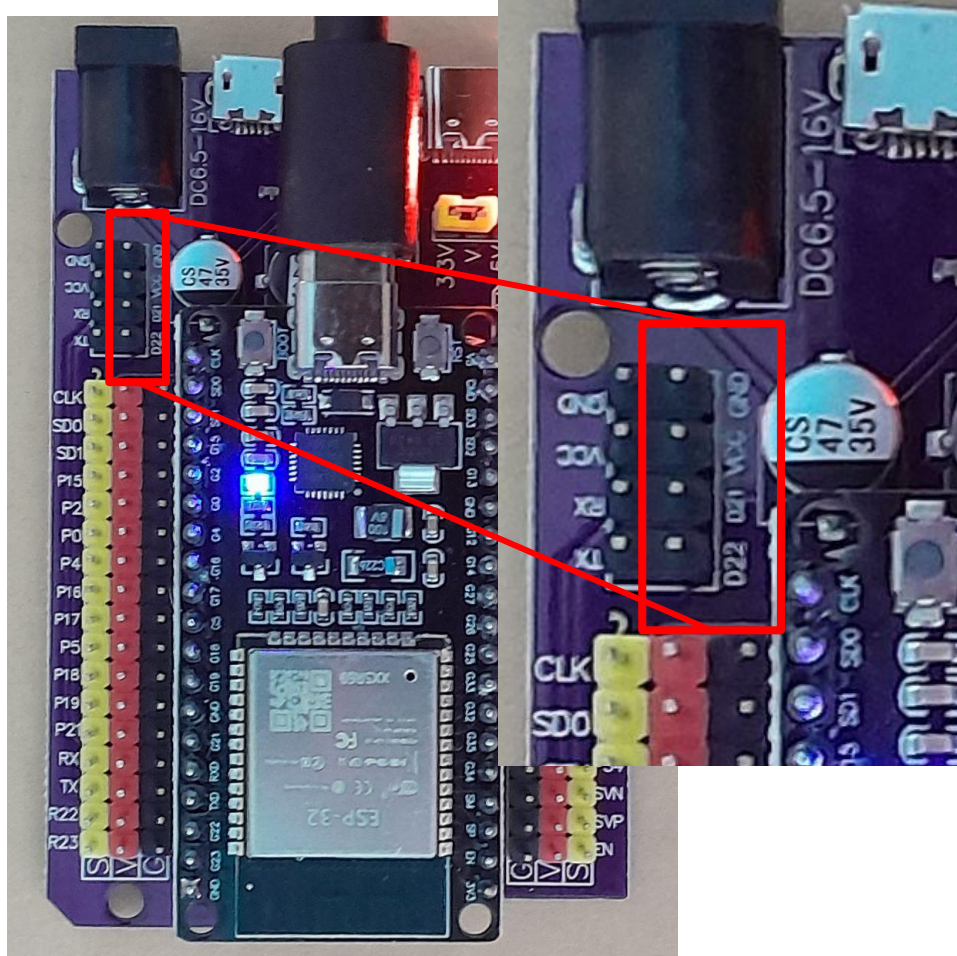
- I²C is presented as group of 4 pins; VCC, GND, SCL, SDA



Topics

- MPR121
12 capacitive-sensing touch pads
- MPU6050
Inertial Measurement Unit: sense orientation & acceleration
- HC-SR04
Ultrasonic Sensor: measure distances with ultrasonic waves
- VL53L0X
Time Of Flight Laser: high speed measurements with laser

MPR121 (12 touch-sensing pins) (mpr121_touch.py)



MPR121 - ESP32

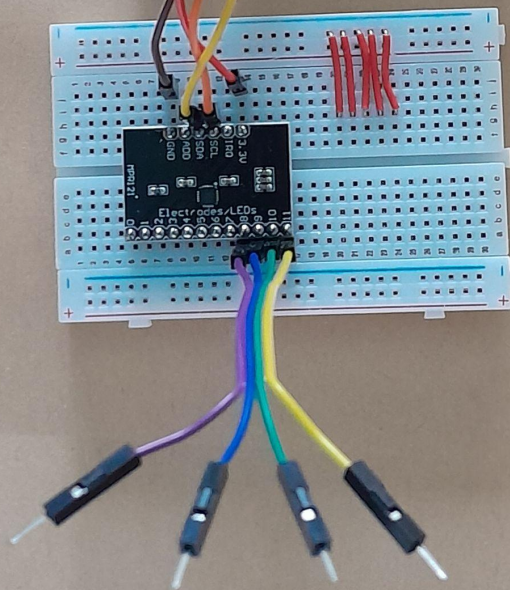
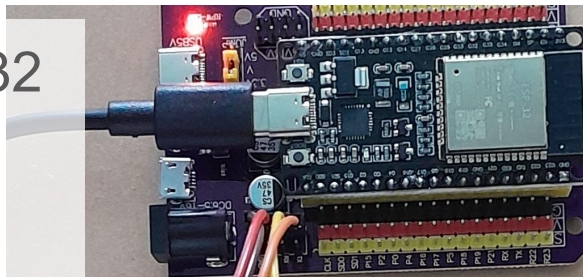
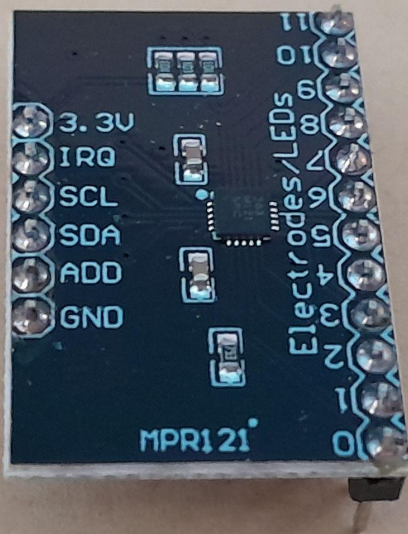
GND	GND
3.3V	VCC
SCL	22
SDA	21



MPR121 (12 touch-sensing pins) (mpr121_touch.py)

MPR121 - ESP32

GND	GND
3.3V	VCC
SCL	22
SDA	21



mpr121_touch.py

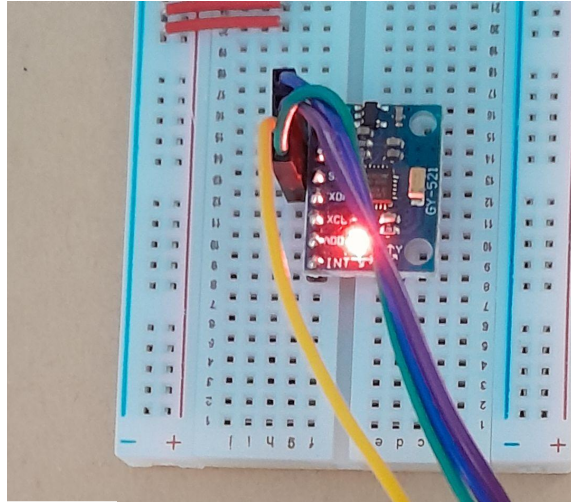
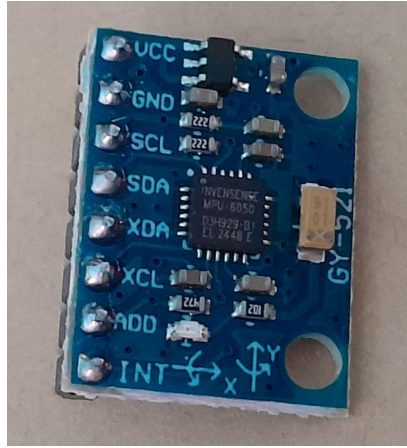
```
1 from mpr121 import MPR121
2 from machine import Pin, I2C
3 import utime
4
5 # Initialize I2C on bus 0 with SDA on GPIO 21 and SCL (
6 i2c = I2C(0, sda=Pin(21), scl=Pin(22))
7
8 # Create an MPR121 object
9 mpr = MPR121(i2c)
10
11 print("Touch sensor ready. Start touching the electrode
12
13 # Main loop to check all electrodes
14 while True:
15     # Get a list of all currently touched electrodes
16     touched_pins = mpr.get_all_states()
17     if len(touched_pins) != 0:
18         print(touched_pins)
19         utime.sleep_ms(100)
20
21 |
```

Shell

```
>>> %Run -c $EDITOR_CONTENT
```

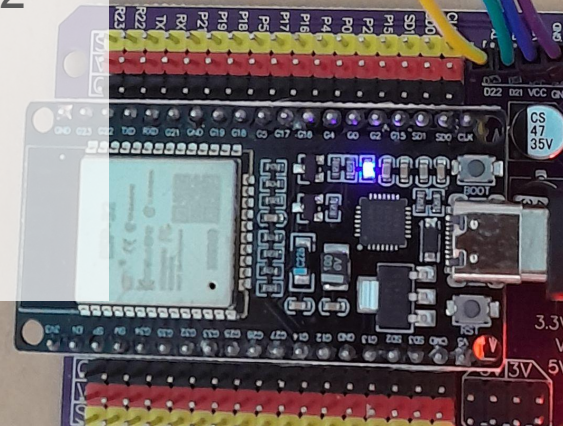
```
MPY: soft reboot
Touch sensor ready. Start touching the electrodes!
[11]
[9]
[10]
[10]
[10]
```

MPU6050 IMU: sense orientation & acceleration (mpu6050.py)



MPU6050 - ESP32

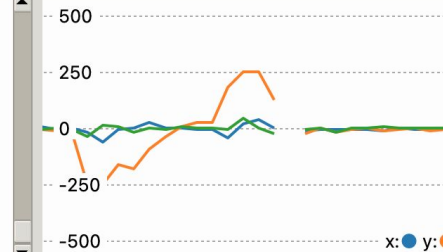
VCC	VCC
GND	GND
SCL	22
SDA	21



```
mpu6050.py x mpu6050_lib.py
7  from time import sleep_ms
8
9  mpu = MPU6050()
10
11  while True:
12      # Accelerometer Data
13      accel = mpu.read_accel_data() # read the accelerometer [ms^-2]
14      aX = accel["x"]
15      aY = accel["y"]
16      aZ = accel["z"]
17      #print("x: " + str(aX) + " y: " + str(aY) + " z: " + str(aZ))
18
19      # Gyroscope Data
20      gyro = mpu.read_gyro_data() # read the gyro [deg/s]
21      gX = gyro["x"]
22      gY = gyro["y"]
23      gZ = gyro["z"]
24      print("x:" + str(gX) + " y:" + str(gY) + " z:" + str(gZ))
25
26      # Rough Temperature
27      temp = mpu.read_temperature() # read the device temperature [degC]
28      # print("Temperature: " + str(temp) + "°C")
```

Shell

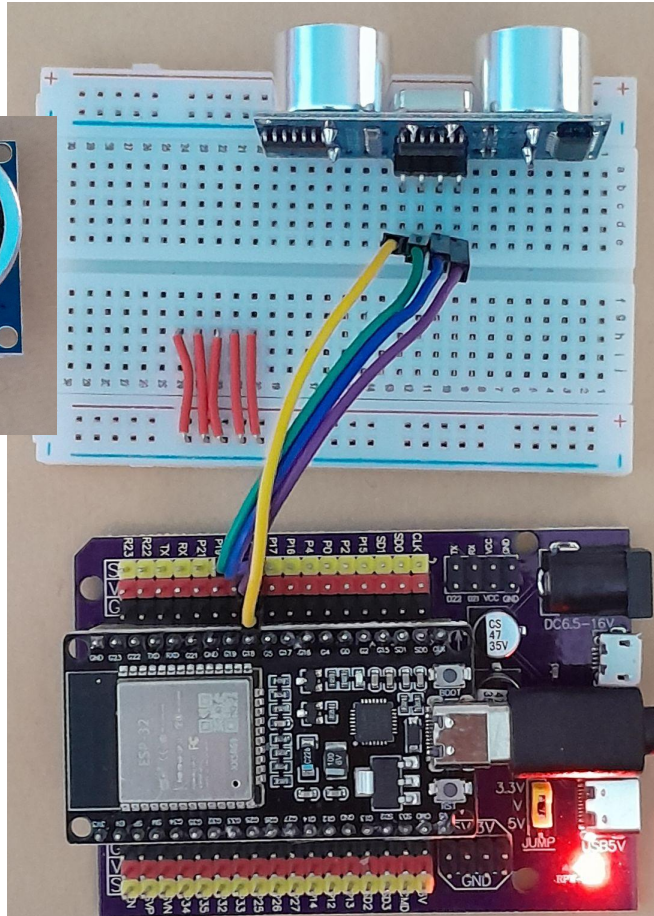
```
at address 0x00, check wiring
x:-12.572519 y:-23.534352 z:-5.152672
x:-7.0305348 y:3.1068702 z:2.572519
x:-8.083969 y:-16.57252 z:-18.503816
x:-3.9694658 y:-4.8015264 z:0.26717558
x:-5.5801524 y:-0.8015267 z:2.5801528
x:-9.480916 y:-10.2213748 z:5.4580156
x:-2.6946564 y:-4.335878 z:-1.2748091
x:-4.7633584 y:-3.0458016 z:-2.5496182
x:-6.374046 y:-10.572519 z:-1.0687023
x:-4.7175576 y:-3.7480916 z:-1.9541985
x:-4.3511452 y:-1.4198474 z:-3.3587788
x:-4.8320612 y:-0.67175576 z:-0.10687023
x:-7.335878 y:1.3053435 z:1.091603
```



HC-SR04 Ultrasonic Sensor (ultrasonic.py)



VCC 5 18 GND



ultrasonic.py

```
1 from hcsr04 import HCSR04
2 from time import sleep
3
4 # Initialize the sensor (adjust pins if you used differ
5 sensor = HCSR04(trigger_pin=5, echo_pin=18)
6
7 while True:
8     try:
9         # Get the distance in centimeters
10        distance_cm = sensor.distance_cm()
11        print('Distance:', distance_cm, 'cm')
12        sleep(.5)
13    except OSError as e:
14        # Handle errors, for example when the measurem
15        print("Sensor error or out of range:", e)
16
17
```

Shell

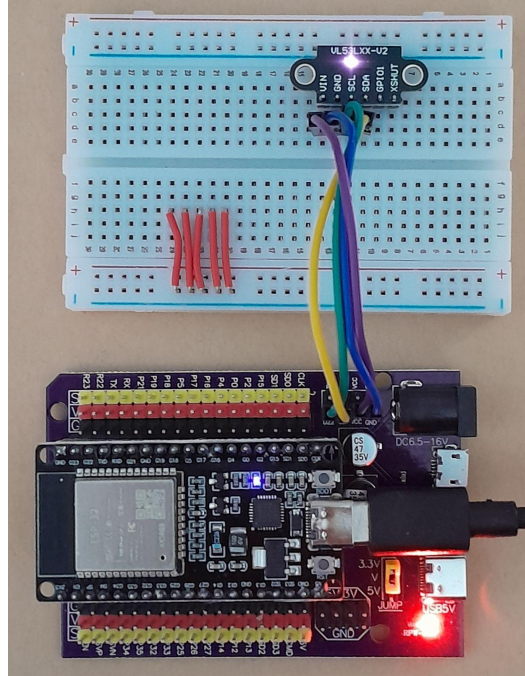
```
Distance: 17.74914 cm
Distance: 17.74914 cm
Distance: 17.74914 cm
Distance: 17.731958 cm
Distance: 17.766324 cm
Distance: 17.731958 cm
Distance: 17.731958 cm
Distance: 17.731958 cm
Distance: 17.731958 cm
Distance: 17.74914 cm
Distance: 17.766324 cm
Distance: 17.731958 cm
Distance: 17.731958 cm
Distance: 17.731958 cm
```

VL53L0X TOF Laser (tof_laser.py)



VL53L0X - ESP32

VCC	VCC
GND	GND
SCL	22
SDA	21

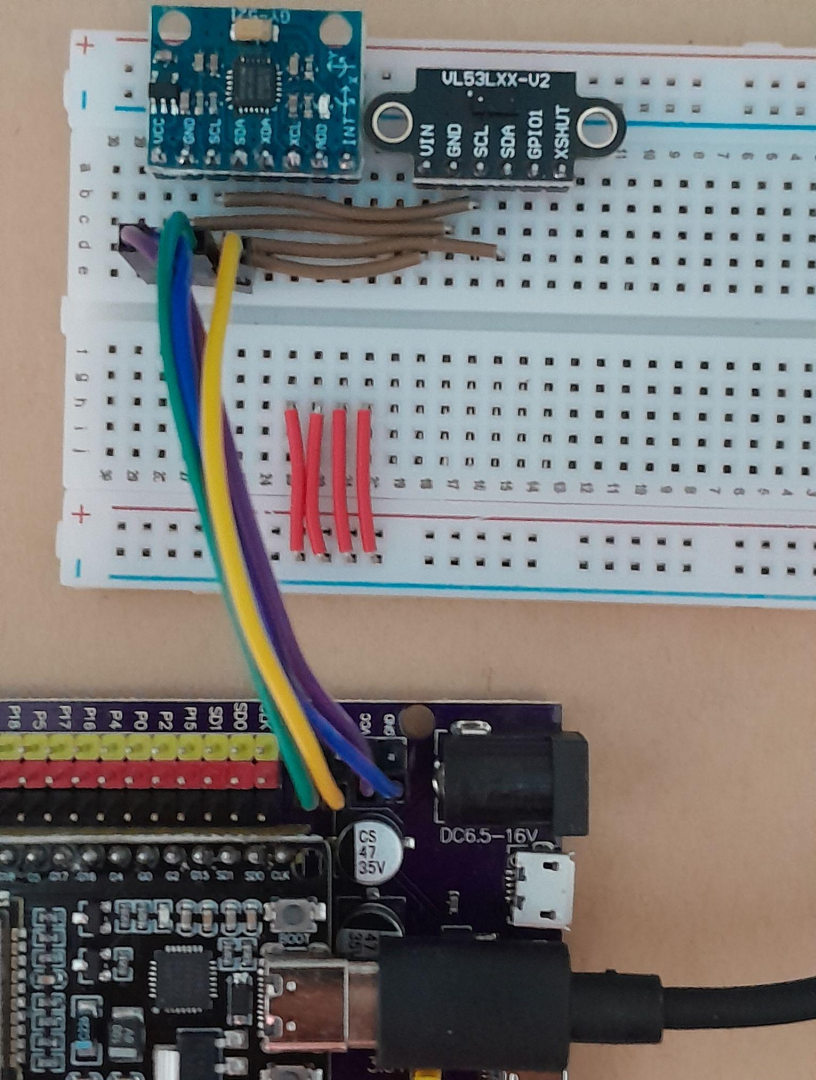


tof_laser.py ×

```
1 import time
2 from machine import Pin, I2C
3 from VL53L0X import VL53L0X
4
5 # Initialize I2C bus
6 i2c = I2C(0, scl=Pin(22), sda=Pin(21))
7
8 # Create a VL53L0X object
9 tof = VL53L0X(i2c)
10
11 # Start continuous ranging
12 tof.start()
13
14 try:
15     while True:
16         # Read the distance in millimeters
17         distance = tof.read()
18         print(f"Distance: {distance} mm")
19         time.sleep_ms(100) # Short delay between readings
20
21 except KeyboardInterrupt:
22     # Stop the sensor and clean up on exit
23     tof.stop()
```

Shell ×

```
Distance: 154 mm
Distance: 153 mm
Distance: 154 mm
Distance: 152 mm
Distance: 157 mm
Distance: 155 mm
Distance: 155 mm
Distance: 154 mm
Distance: 148 mm
Distance: 148 mm
Distance: 147 mm
Distance: 149 mm
Distance: 149 mm
Distance: 147 mm
```



Connecting multiple I²C devices

VL53L0X SCL SDA VCC GND

MPU6050 SCL SDA VCC GND

ESP32 22 21 VCC GND

Session 1-3 prompt

Design something that turns everyday moments into a show.

If you are dry on ideas...

- A wacky music instrument control panel
- A control panel that works with human movement
- A control panel that looks or acts like a creature

